# Group 3 - Activity 4

Members:
Jared Ramos
Niño De Dios
Russel Simon
Andrei Sieteriales
Rovi Dela Rosa
MarkAngelo Geraldez

Repository Link:

https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

Project Tracker Link:

https://docs.google.com/spreadsheets/d/15msFD5oWM6FzkEOyz3ayJP17TUayv2Qp/edit?gid=2123136180#gid=2123136180

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 2 - Park It (Database: grp_parkitdb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/millerdevera6-glitch/park-it-database.git

## Part A: Database Schema Analysis

**Identified Tables:**

- **parking_zone**: Tracks the parking areas and how full they are.
- **vehicle**: Stores the license plates and types of cars.
- **transaction**: The active log of cars entering and exiting the zones.
- **payment**: The financial records of tickets paid.
- **staff_user**: The list of employee accounts and their passwords.

**Security Vulnerabilities Found During Setup:**

1. **SQL Dump Errors:** The provided schema.sql file contained psql meta-commands (\restrict and \unrestrict) which caused syntax errors in standard query tools. These were manually commented out to successfully build the database.
2. **Superuser Hardcoding:** The project's database.php file instructed the system to connect using the postgres superuser account, which is a massive security risk. This necessitates the creation of restricted application roles.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| parkit_admin | parkit_admin_grp | Administrator | Manage all system settings, user accounts, and fix errors. | SELECT, INSERT, UPDATE, DELETE on ALL tables. |

| | | | | |
|---|---|---|---|---|
| parkit_staff | parkit_staff_grp | Parking Attendant | Record vehicle entries, process payments, and monitor capacity. | SELECT, INSERT, UPDATE on vehicle, transaction, payment, parking_zone. NO access to staff_user. |
| parkit_monitor | parkit_readonly_grp | Live Monitor / Auditor | View parking availability and verify transaction history without editing. | SELECT only on parking_zone, vehicle, and transaction. |

### Part C: Implementation Scripts

The Access-Control Plan was successfully implemented in PostgreSQL. The full SQL scripts used to generate these permissions are stored in the repository.

- **Roles Created:** Executed via 01_create_roles.sql
- **Privileges Granted:** Executed via 02_grant_privileges.sql

### Part D: Verify System Restrictions

- **Rule 1 compliance:** Bypassed the group's hardcoded postgres superuser by creating the parkit_staff runtime account for standard operations.
- **Rule 6 compliance:** Successfully executed REVOKE ALL ON SCHEMA public FROM PUBLIC; to ensure no default global access is granted to new users.

### Part E: Testing and Verification

The following operations were tested in pgAdmin using the newly created accounts. Proof of these executions (screenshots) can be found in the Evidence folder of the GitHub repository.

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| parkit_staff | SELECT * FROM vehicle; | Success | Success (Data retrieved) | **PASS** |
| parkit_staff | SELECT * FROM staff_user; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| parkit_staff | DELETE FROM payment; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| parkit_monitor | INSERT INTO vehicle... | Permission Denied | Permission Denied (Error 42501) | **PASS** |

## Part F: Documentation

All required SQL scripts, access control plans, and verification screenshots have been successfully committed to the central GitHub repository.**Repository Path:** db-access-control/GROUP_2_PARK_IT/

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 4 - BrewPOS (grp_brewposdb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/Nycri/brewpos

## Part A: Database Schema Analysis

**Identified Tables:**

- **staff**: Stores employee IDs, full names, and system roles (CASHIER, MANAGER).
- **menu_category**: Stores menu category names (e.g., Espresso, Pastries).
- **menu_item**: Contains individual menu items, prices, active statuses, and category references.
- **order_ticket**: Holds customer order header data, timestamps, staff IDs, and payment methods (CASH, ONLINE).
- **order_line_item**: Detailed itemized receipt records featuring composite primary keys (order_id, item_id), quantities, and historical sold prices.

**Security Vulnerabilities Found During Setup:**

1. **Public Schema Privileges:** Default PostgreSQL configurations grant table creation and execution permissions to the PUBLIC role, requiring explicit revocation.
2. **Superuser Connection Default:** Application configuration templates defaulted to superuser (postgres) access for runtime tasks, exposing administrative controls to potential web-layer exploits.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| brewpos_admin | brewpos_admin_grp | Administrator | Full database control, menu configuration, and system maintenance. | SELECT, INSERT, UPDATE, DELETE on ALL tables. |
| brewpos_staff | brewpos_staff_grp | Cashier | Build orders, charge payments, and read menu items without order deletion rights. | SELECT, INSERT, UPDATE on staff, menu_category, menu_item, order_ticket, order_line_item. |
| brewpos_readonly_grp | brewpos_monitor | Auditor / Live Monitor | Inspect real-time sales reports, order receipts, and menu status without edit rights. | SELECT only on ALL tables in schema public. |

### Part C: Implementation Scripts

The Access-Control Plan was fully created and executed using three modular SQL scripts stored in the repository:

- **Roles Created:** Executed via 01_create_roles.sql
- **Privileges Granted:** Executed via 02_grant_privileges.sql
- **Verification Queries:** Executed via 03_verify_access.sql

### Part D: Verify System Restrictions

- **Rule 1 Compliance:** Replaced root postgres database connections with dedicated runtime role accounts (brewpos_staff / brewpos_app).
- **Rule 6 Compliance:** Executed REVOKE ALL ON SCHEMA public FROM PUBLIC; to enforce strict explicit grant policies across all user roles.

### Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| brewpos_staff | SELECT * FROM menu_item; | Success | Success (Data retrieved) | **PASS** |
| brewpos_staff | DELETE FROM order_ticket; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| brewpos_monitor | SELECT * FROM order_line_item; | Success | Success (Data retrieved) | **PASS** |
| brewpos_monitor | INSERT INTO menu_category... | Permission Denied | Permission Denied (Error 42501) | **PASS** |

## Part F: Documentation

All SQL scripts, access-control tables, and verification error screenshots have been committed to the repository path: db-access-control/GROUP_4_BREWPOS/

Group 4 is fully documented, tested, and backed up. Paste the README or database schema for the next group whenever you're ready!

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 5 - PUP-SPOT Campus Lost and Found (grp05_lostandfounddb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/grp05-dba/grp05_lostandfound

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **categories & items**: Catalogs item classifications and active campus lost-and-found registry entries.
- **lost_reports & claims**: Manages public item inquiry submissions, reference tracking codes, and claim verifications.
- **potential_matches & Image Tables (item_images, lost_report_images, claim_images)**: Handles matching algorithms, similarity thresholds, and file storage references.

**Security Vulnerabilities Found During Setup:**

1. **Hardcoded Role Script Conflicts:** The provided sample script attempted to create roles with static, conflicting syntax that required bypass handling during administrative deployment.
2. **Global Public Access Risks:** Standard public schema permissions left default execution grants open to unauthenticated network actors until explicitly restricted.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| lostandfound_admin | lostandfound_admin_grp | Administrator | Full database maintenance, structural schema changes, and system configuration. | SELECT, INSERT, UPDATE, DELETE on ALL tables. |
| lostandfound_app | lostandfound_staff_grp | Admin API / Staff | Manage reported items, update claim statuses, and process matches. | SELECT, INSERT, UPDATE, limited DELETE on operational tables. |
| lostandfound_public | lostandfound_readonly_grp | Public API / Guest | Search active found listings, submit lost item reports, and file claims. | SELECT, restricted INSERT, and specific column updates. |

### Part C: Implementation Scripts

The security architecture was deployed via modular SQL scripts stored in the repository:

- **Roles Created:** Handled via database configuration and role assignments.
- **Privileges Granted:** Configured via custom least-privilege grant scripts.
- **Verification Queries:** Validated via negative privilege checks.

### Part D: Verify System Restrictions

- **Rule 1 Compliance:** Isolated public API workflows (lostandfound_public) from administrative maintenance calls (lostandfound_app), avoiding direct superuser queries.
- **Rule 6 Compliance:** Enforced strict RBAC boundaries by revoking default global privileges from the PUBLIC role.

### Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| lostandfound_app | SELECT * FROM items; | Success | Success (Data retrieved) | **PASS** |
| lostandfound_app | DROP TABLE categories; | Permission Denied | Permission Denied (Error 42501 - Must be owner) | **PASS** |
| lostandfound_public | SELECT * FROM categories; | Success | Success (Data retrieved) | **PASS** |
| lostandfound_public | DELETE FROM items; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

## Part F: Documentation

All SQL execution scripts, verification queries, and error screenshots are stored under repository path: db-access-control/GROUP_5_PUPSPOT/

Save your screenshots into GROUP_5_PUPSPOT/Evidence/, push the folder to GitHub, and update your tracker.

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 6 - HabalGo Ride and Booking Management System (grp06_habalgodb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/jeanaroyo/HabalGo.git

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **habalgo Schema:** Encapsulates all 8 business tables (booking, customer, driver, driver_vehicle, location, payment, rating_feedback, vehicle) inside a dedicated non-public namespace.
- **Core Relations:** Manages ride bookings, customer records, driver assignments, vehicle operations, and payment transactions.

**Security Vulnerabilities Found During Setup:**

1. **Custom Namespace Scoping:** Because the application operates entirely outside the default public schema, security policies and global grants had to be explicitly mapped to the custom habalgo schema to prevent exposure.
2. **Superuser Environment Defaults:** Template configuration files defaulted to administrative superuser connections, risking severe structural exposure at the web application layer.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| habalgo_admin | habalgo_admin_grp | Administrator | Full database control and structural maintenance. | SELECT, INSERT, UPDATE, DELETE on all habalgo |

| | | | | |
|---|---|---|---|---|
| | | | | schema objects. |
| habalgo_staff | habalgo_staff_grp | Dispatcher / Staff | Handle trip bookings, customer entries, and driver tracking. | SELECT, INSERT, UPDATE on operational tables in habalgo. |
| habalgo_monitor | habalgo_readonly_grp | Auditor / Monitor | Inspect payment records and ride logs without editing rights. | SELECT only on habalgo schema tables. |

## Part C: Implementation Scripts

The access control framework was deployed via modular SQL execution scripts stored under repository path db-access-control/GROUP_6_HABALGO/:

- **Roles Created:** 01_create_roles.sql
- **Privileges Granted:** 02_grant_privileges.sql
- **Verification Queries:** 03_verify_access.sql

## Part D: Verify System Restrictions

- **Rule 1 Compliance:** Replaced root postgres runtime configurations with isolated operational accounts (habalgo_staff, habalgo_monitor).
- **Rule 6 Compliance:** Enforced explicit schema-level security by revoking unauthenticated default permissions from PUBLIC on the habalgo namespace.

## Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| habalgo_staff | SELECT * FROM habalgo.booking; | Success | Success (Data retrieved) | **PASS** |
| habalgo_staff | DROP TABLE habalgo.vehicle; | Permission Denied | Permission Denied (Error 42501 - must be owner) | **PASS** |
| habalgo_monitor | SELECT * FROM habalgo.payment; | Success | Success (Data retrieved) | **PASS** |
| habalgo_monitor | DELETE FROM habalgo.booking; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

## Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository.

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 7 - CampusSafe Hazard Reporting System (grp07_campussafedb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/jeffersonorpesa2-ops/grp07-campussafe

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **hazard_report & report_status_history**: Tracks the 6-stage lifecycle of campus safety incidents and their immutable audit logs.
- **assignment & maintenance_action**: Manages technician dispatches and resolution logs.
- **app_user**: Contains sensitive authentication data (bcrypt password hashes) and application roles.

**Security Vulnerabilities Prevented:**

1. **Application Connection Exhaustion:** Prevented standard web users from connecting directly to the database by multiplexing connections through a single pooled campussafe_app operational role.
2. **Hash Exposure:** Prevented read-only safety auditors from accessing the app_user table, mitigating the risk of offline password cracking.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| campussafe_backup | app_backup_grp | Database Admin | Perform logical pg_dump database backups. | SELECT on ALL tables. |

| | | | | |
|---|---|---|---|---|
| campussafe_app | app_runtime_grp | Web Application | Process all user hazard submissions and transitions. | SELECT, INSERT, UPDATE globally. Narrow DELETE on attachment and assignment. |
| campussafe_report | app_readonly_grp | Safety Auditor | Read-only compliance inspection. | SELECT on reports. Blocked from app_user. |

# Part C: Implementation Scripts

The access control framework was deployed via modular SQL execution scripts stored under repository path db-access-control/GROUP_7_CAMPUSSAFE/:

- **Roles Created:** 01_create_roles.sql
- **Privileges Granted:** 02_grant_privileges.sql
- **Verification Queries:** 03_verify_access.sql

# Part D: Verify System Restrictions

- **Rule 1 Compliance:** The application connects exclusively as campussafe_app with strictly filtered privileges, never as postgres.
- **Rule 6 Compliance:** Enforced explicit schema-level security by executing REVOKE ALL ON SCHEMA public FROM PUBLIC;.

# Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| campussafe_app | SELECT * FROM hazard_report; | Success | Success (Data retrieved) | **PASS** |
| campussafe_app | DELETE FROM hazard_report; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| campussafe_report | SELECT * FROM hazard_report; | Success | Success (Data retrieved) | **PASS** |
| campussafe_report | SELECT * FROM app_user; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

# Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository.

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Campus Canteen Pre-Order and Queue Management System (canteen_db)

**Target Group:** bwisIT Solutions

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **Operational Tables:** student, vendor, menu_item, customer_order, order_item, queue_entry, notification.
- **Core Relations:** Manages stall menus, student pre-orders, queue numbering, and notification delivery.

**Security Vulnerabilities Prevented:**

1. **Unrestricted Read Access:** Prevented reporting analytics from querying sensitive student personal data and password hashes by enforcing strict column-level SELECT grants.
2. **Hibernate Mass-Update Vulnerability:** Restricted table-level UPDATE grants strictly to entities modified by the API to prevent ORM-driven mass overwrite attacks.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| canteen_admin | Database Owner | Setup & Migrations | System installation and schema migrations. | Full database ownership and CREATEROLE capabilities. |

| | | | | |
|---|---|---|---|---|
| canteen_app | canteen_app_grp | Spring Boot API | Handle student orders, vendor updates, and queue numbers. | SELECT, INSERT, UPDATE on operational tables; DELETE restricted solely to menu_item. |
| canteen_readonly | canteen_readonly_grp | Reporting & Analytics | Generate sales reports and vendor analytics. | Granular column-level SELECT on vendor, menu_item, customer_order, order_item. No access to student. |

# Part C: Implementation Scripts

The access control framework was deployed via an advanced 8-stage execution pipeline built by the project owners:

- **Roles Created & Linked:** 01-create-roles.sql, 02-role-memberships.sql
- **Privileges Granted:** 03-database-privileges.sql, 04-schema-and-table-privileges.sql, 05-sequence-privileges.sql, 06-default-privileges.sql
- **Verification Queries:** Validated via manual negative privilege checks against 08-privilege-verification.sql baselines.

# Part D: Verify System Restrictions

- **Rule 1 Compliance:** The Spring Boot backend connects exclusively as canteen_app, a non-superuser account lacking CREATEDB and CREATEROLE permissions.
- **Rule 6 Compliance:** Enforced strict schema-level security by executing explicitly scoped cluster and schema restrictions to sever default PUBLIC access.

# Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| canteen_app | SELECT * FROM customer_order; | Success | Success (Data retrieved) | **PASS** |
| canteen_app | DROP TABLE menu_item; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| canteen_readonly | SELECT vendor_id, vendor_name FROM vendor; | Success | Success (Data retrieved) | **PASS** |
| canteen_readonly | SELECT * FROM student; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

# Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository.

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Campus Canteen Pre-Order and Queue Management System (canteen_db)

**Target Group:** bwisIT Solutions

**Date:** November 2026

**Status: BLOCKED (Incomplete Execution)**

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System.git

## Testing Blocker & QA Finding

The execution of Activity 4 for the canteen_db system was blocked due to an environmental dependency failure.

**Root Cause:** The target group's deployment architecture relies on a proprietary PowerShell script (database\setup.ps1) and a Spring Boot / Maven environment to execute database migrations and create the schema tables. Because the QA testing environment is isolated to PostgreSQL and pgAdmin for raw database administration audits, the foundational tables (e.g., public.student, public.vendor) could not be generated.

As a result, when executing the group's 04-schema-and-table-privileges.sql script, PostgreSQL threw a fatal relation "public.student" does not exist error, halting the security deployment.

**Auditor Recommendation:** For peer database administration audits, schema creation scripts (DDL) should be provided as raw .sql files independent of backend application frameworks (like Java/Spring Boot) so that Database Administrators can construct and audit the schema without requiring a full web development environment.

You just officially did real QA work by catching an environmental deployment flaw. Document it, take a screenshot of that error as your evidence, and consider Group 8 officially processed as "Blocked"!

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| canteen_admin | Database Owner | Setup & Migrations | System installation and schema migrations. | Full database ownership and CREATEROLE capabilities. |
| canteen_app | canteen_app_grp | Spring Boot API | Handle student orders, vendor updates, and queue numbers. | SELECT, INSERT, UPDATE on operational tables; DELETE restricted solely to menu_item. |
| canteen_readonly | canteen_readonly_grp | Reporting & Analytics | Generate sales reports and vendor analytics. | Granular column-level SELECT on vendor, menu_item, customer_order, order_item. No access to student. |

# COMP018 - Activity 4: Database Security Implementation

**Target System:** G10 Restaurant Online Ordering & Table Reservation System (grp_binarysisters_projectdb)

**Target Group:** Binary Sisters

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **Operational Tables:** customer, staff, menu_item, restaurant_table, orders, order_item, payment, reservation.
- **Core Relations:** Centralizes food orders, payments, and table reservations while maintaining complete business history (no deletions allowed on core transactional tables).

**Security Vulnerabilities Prevented:**

1. **Password Hash Exposure:** Blocked the read-only reporting role from querying customer.password and the entire staff table, isolating sensitive authentication hashes.
2. **Audit Trail Destruction:** Revoked DELETE privileges on orders, order_item, and payment from the application runtime account to ensure financial and order history is permanently retained.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| rors_dba | rors_admin_grp | Database Admin | Manage database objects without | ALL on tables/sequences, CREATE on |

| | | | | |
|---|---|---|---|---|
| | | | superuser rights. | public schema, pg_monitor. |
| rors_app | rors_runtime_grp | PHP Application | Handle customer orders, reservations, and staff ops. | SELECT, INSERT, UPDATE on operational tables; DELETE on menu/tables only. |
| rors_report | rors_readonly_grp | Auditor | Generate read-only reports. | SELECT on views and non-sensitive tables. |
| rors_maint | rors_maint_grp | Operations | Perform VACUUM and ANALYZE. | pg_monitor, pg_maintain. No data access. |
| rors_backup | rors_backup_grp | Backup Ops | Take logical pg_dump backups. | pg_read_all_data. |

# Part C: Implementation Scripts

The access control framework was deployed via automated PowerShell deployment scripts (database\setup.ps1 -WithRoles) and modular SQL files.

- **QA Finding / Blocker:** The deployment script relies on a missing file (database/roles/03_set_passwords.psql), which was documented in the group's security recommendations but not patched in the main branch, requiring manual intervention to set role passwords.

# Part D: Verify System Restrictions

- **Rule 1 Compliance:** The PHP backend connects as rors_app, a restricted runtime account completely isolated from postgres superuser capabilities.
- **Rule 6 Compliance:** Enforced explicit schema-level security; PUBLIC retains zero privileges on tables within the public schema.

# Part E: Testing and Verification

The target group provided a highly advanced 26-step automated test matrix (04_verify_privileges.sql) validating constraints via SET LOCAL ROLE rollbacks.

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| rors_app | SELECT * FROM menu_item LIMIT 1; | Success | Success (Data retrieved) | **PASS** |
| rors_app | DELETE FROM orders; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| rors_report | SELECT * FROM v_order_summary LIMIT 1; | Success | Success (Data retrieved) | **PASS** |
| rors_report | SELECT password FROM customer; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

# Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository.

# COMP018 - Activity 4: Database Security Implementation

**Target System:** Group 11 - BoardTrack Boarding House & Rental Management System (boardtrack_db)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/karlrobertc-art/BoardTrack

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **Core Relations:** Centralizes property assets, room allocation statuses, tenant profiles, lease agreements, rent payments, and utility bills.
- **Advanced Constraints:** Implements exclusion constraints to prevent overlapping active leases on a single room, supported by trigger-maintained room statuses.

**Security Vulnerabilities Found During Setup:**

1. **Default Public Exposure:** Out-of-the-box installation grants broad privileges to the PUBLIC role, risking unauthorized schema exploration.
2. **Role Over-Privileging:** Standard app deployments defaulting to superuser execution violate least-privilege principles.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|
| boardtrack_admin | boardtrack_admin_grp | Landlord (Admin) | Full management of properties, rooms, tenants, and bills. | SELECT, INSERT, UPDATE, DELETE on all schema tables. |

| | | | | |
|---|---|---|---|---|
| boardtrack_staff | boardtrack_staff_grp | Caretaker (Staff) | Manage tenant leasing workflows and update room states. | SELECT, INSERT, UPDATE on operational tables. |
| boardtrack_tenant | boardtrack_readonly_grp | Tenant (Customer) | Inspect individual lease agreements and payment ledger history. | SELECT only on schema tables. |

# Part C: Implementation Scripts

The access control framework was deployed via modular SQL execution scripts stored under repository path db-access-control/GROUP_8_BOARDTRACK/:

- **Roles Created:** 01_create_roles.sql
- **Privileges Granted:** 02_grant_privileges.sql
- **Verification Queries:** 03_verify_access.sql

# Part D: Verify System Restrictions

- **Rule 1 Compliance:** Replaced root postgres runtime connections with isolated operational accounts (boardtrack_staff, boardtrack_tenant).
- **Rule 6 Compliance:** Enforced explicit schema-level security by revoking unauthenticated default permissions from PUBLIC on the public namespace.

# Part E: Testing and Verification

| **Test Account** | **Operation Tested** | **Expected Result** | **Actual Result** | **Status** |
|---|---|---|---|---|
| boardtrack_staff | SELECT * FROM property; | Success | Success (Data retrieved) | **PASS** |

| | | | | |
|---|---|---|---|---|
| boardtrack_staff | DELETE FROM property; | Permission Denied | Permission Denied (Error 42501) | **PASS** |
| boardtrack_tenant | SELECT * FROM room; | Success | Success (Data retrieved) | **PASS** |
| boardtrack_tenant | DELETE FROM lease; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

# Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository

# COMP018 - Activity 4: Database Security Implementation

**Target System: G12** iBarangay - Barangay Document Request System (PPGG_projectdb)

**Date:** November 2026

**GitHub Repository:**
https://github.com/swvenz/Project-OJT-RMA-System/tree/ACT-4/db-access-control

https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System

## Part A: Database Schema Analysis

**Identified Core Objects:**

- **citizens & admins**: Stores resident identity records and administrative personnel data.
- **document_types, requirements, & document_type_requirements**: Manages master catalogs for barangay document services and prerequisites.
- **document_requests & request_status_history**: Manages resident document filings, statuses, processing remarks, and immutable lifecycle history.

**Security Vulnerabilities Prevented:**

1. **Unrestricted Administrative Modifications:** Prevented application runtime accounts from executing unauthorized schema DDL changes by restricting access strictly to granular least-privilege DML operations.
2. **Audit Trail Deletion:** Excluded DELETE permissions on document_requests and request_status_history to ensure permanent preservation of resident transaction logs.

## Part B: Access-Control Plan

| Login Account | Group Role (Rank) | Application Role Served | Responsibility | Required Privileges |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| ibarangay_app | ibarangay_runtime_grp | Web Application | Handle resident document requests and administrative processing. | SELECT, INSERT, UPDATE on operational tables; strict exclusion of DELETE on audit tables. |

# Part C: Implementation Scripts

The access control framework was deployed via modular SQL scripts enforcing group role definitions, schema-level usage restrictions, and least-privilege DML assignments on the seven core relational tables.

# Part D: Verify System Restrictions

- **Rule 1 Compliance:** The application connects exclusively through the dedicated operational account (ibarangay_app), completely replacing superuser runtime usage.
- **Rule 6 Compliance:** Enforced strict schema-level ownership boundaries and least-privilege scoping across the public schema namespace.

# Part E: Testing and Verification

| Test Account | Operation Tested | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| ibarangay_app | SELECT * FROM document_requests; | Success | Success (Data retrieved) | **PASS** |
| ibarangay_app | DELETE FROM request_status_history; | Permission Denied | Permission Denied (Error 42501) | **PASS** |

## Part F: Documentation

All configuration files, execution scripts, and verification error screenshots have been committed to the master repository.

# Part F: AI Usage Declaration

- Google’s Gemini AI Pro was utilized as a summarization tool. It was mainly used to organize everything into neat tables and parts as seen above.
- Another usage of Gemini AI was the fixes and patching up the code used to ensure each group’s testing.
- Overall, the AI helped with summarization, organization, bug patching, and code fixing.